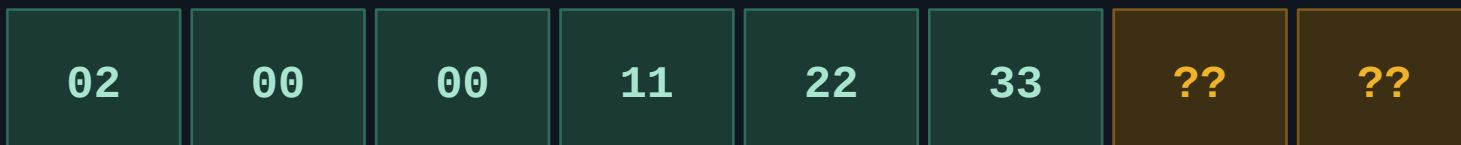


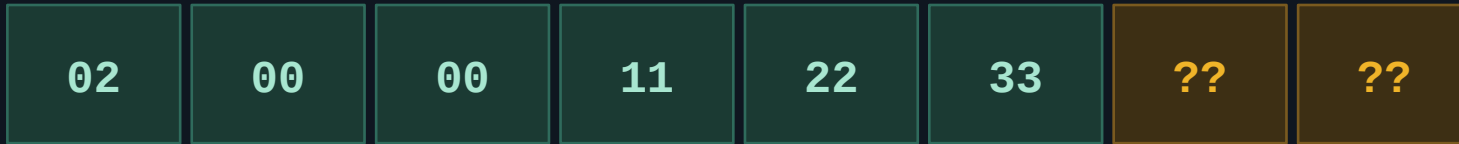
Eight bytes from a six-byte field

Comparing six-byte addresses with six-byte and eight-byte reads



Six address bytes and two extra bytes in this experiment

Six bytes of address, eight of storage



six bytes identify the address

two extra bytes

This experiment uses eight-byte arrays. Only the first six bytes identify each address.

Both versions compare six address bytes. They read different amounts.

Memory order: increasing addresses, left to right

The wide comparison

```
bool equal_64bits(const unsigned char* a,
                  const unsigned char* b) {
    std::uint64_t x, y;
    std::memcpy(&x, a, 8);
    std::memcpy(&y, b, 8);
    return ((x ^ y) << 16) == 0;
}
```

One eight-byte read per input

eight bytes copied into each integer

XOR marks the differences

equal bytes cancel to zero

The shift drops the top sixteen bits

the two extra bytes leave the result

A zero result means the six address bytes match.

The split comparison

```
std::uint32_t x, y;  
std::uint16_t u, v;  
std::memcpy(&x, a, 4);  
std::memcpy(&y, b, 4);  
std::memcpy(&u, a + 4, 2);  
std::memcpy(&v, b + 4, 2);  
return ((x ^ y) | (u ^ v)) == 0;
```

Four bytes, then two, from each input.

This reads only six bytes from each input. Like the eight-byte version, it has no data-dependent branch in this build.

Both versions are branchless. Which one is faster?

The eight-byte version was faster

Nanoseconds per pair. Lower is faster.

Comparison	Nanoseconds per pair
Wide eight	0.35
Split four + two	0.81
memcmp six	0.58

All three use the same eight-byte arrays.

262,144 matching pairs, 7 samples. GCC 13.3, -O3, default x86-64 target, AMD EPYC 9V74.
Median batch throughput on a shared host. No CPU pinning or frequency control.

The wide loop groups whole inputs

Two 16-byte loads bring in two pairs

a0: 8 bytes

b0: 8 bytes

a1: 8 bytes

b1: 8 bytes

Two unpack instructions group the A inputs and the B inputs

a0

a1

b0

b1

```
punpckhqdq xmm1, xmm4
punpcklqdq xmm0, xmm4
pxor        xmm0, xmm1
psllq       xmm0, 16
```

**XOR and shift handle
both pairs at once.**

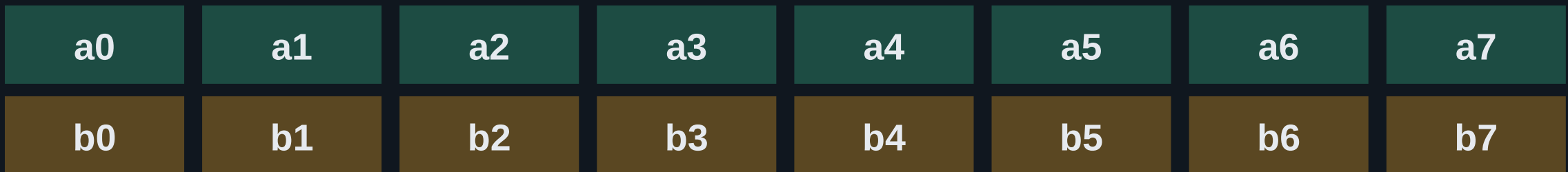
The split loop gathers smaller pieces

Each input splits into 4 bytes, 2 bytes, and 2 unused bytes



More loads and repeated unpacking collect the pieces

Two-byte pieces after the unpacking chain



The four-byte pieces form separate vectors.

GCC 13.3, -O3. scan<3> in benchmark/bench-O3.asm. Each bottom cell is 2 bytes.

More work for the same eight pairs

Repeating loop body, normalized to eight compared pairs

Generated work	Wide	Split
16-byte load instructions	8	16
Shuffle / unpack instructions	12	52

The split loop does more work arranging the data.

Counts include result handling. They are not cycle estimates.
Wide loop: 2 pairs per iteration. Split loop: 8 pairs per iteration.

**Reading fewer bytes didn't make
this comparison faster.**